cursor-hls-kb

HLS Knowledge Base — User Tutorial

Cursor AI × Vitis HLS × Knowledge Base — Automated Design Workflow

AICOFORGE · v1.0 · CC BY-NC 4.0

System overview

Centralized Knowledge Base

PostgreSQL stores UG1399 official rules (287) + user-defined rules (100+) to track application performance

Cursor AI Automation

Connect to the Vitis-HLS host through Remote SSH, query the rules for design, and automatically write them into the Knowledge Base after successful synthesis.

Precise Rollback mechanism

_rollback_info metadata supports iteration/project-level accurate restoration of rules_effectiveness statistics

Lightweight API access

FastAPI (port 8000) provides endpoints such as rule query, iteration record, synthesis results, etc., and can be called directly from the intranet

→ *Please see README "Features" and "Overview" chapters for details*

System architecture and machine configuration

HLS01

Knowledge Base host

PostgreSQL + FastAPI
Administrator installation settings

HLS02

Vitis-HLS host

Users use Cursor
Agency HLS Design

HLS03

Vitis-HLS host

Users use Cursor
Agency HLS Design

Access method

1. Cursor AI Agent → SSH → Vitis-HLS host → Intranet → FastAPI (port 8000) read and write Knowledge Base
2. DBeaver → SSH Tunnel / direct connection → PostgreSQL (port 5432) — administrator read-write, user read-only

→ Please refer to README "System Architecture" and "Machine Configuration" chapters for details.

Administrator installation settings (HLS01)

1

Environmental preparation

Ubuntu 22.04+, execute env_setup.sh to install Docker / Python3 / pip3, etc.

2

Modify environment variables

Modify DB_ADMIN / DB_ADMIN_PASS at the top of setup.sh (must be modified before first use)

3

Execute setup.sh

Automatically create DB schema, container, and import 391 rules (287 official + 104 custom)

4

verify

curl http://localhost:8000/health Confirm that the API is normal and verify the number of rules

Note: DB_USER cannot be set to user (PostgreSQL reserved word); the default is hls_user.

→ For details, please refer to the README "Administrator Quick Start", "Prerequisites" and "System Initialization"

Database Tools - Administrator Quick Guide (HLS01)

1 backup_restore.py

Backup/Restore Complete Database

- **backup**
  Generate SQL dump + JSON metadata to util/backups/
- **list**
  List all existing backup files and timestamps
- **restore <file.sql>**
  Restore from the specified dump; the current DB will be overwritten. You need to enter yes to confirm.

2 reset_database.py

Clear all data (keep schema)

- **--stats**
  First check the current number of records in each table without changing the database.
- **(no parameters)**
  Clear projects / iterations / synthesis_results / rules_effectiveness and other tables; schema is retained

3 logger-rollback.py

Accurate rollback of a single iteration (two phases)

- **logger --project --iteration**
  Read _rollback_info to generate YAML logs (without changing DB)
- **rollback --dry-run <log>**
  Preview: List of rules_effectiveness and deleted items that will be restored
- **rollback <log>**
  Single transaction execution; if any step fails, the entire transaction will be rolled back

Environment variables are required before execution: `DB_ADMIN=admin` · `DB_ADMIN_PASS=admin_passwd` .env & .bashrc are automatically written by setup.sh. If it is not loaded, the database cannot be connected.

→ Please refer to the README "Backup and Restore", "Database Tool" and "Rollback Mechanism" chapters for details.

User environment settings

1

Edit hls-env.conf

Set KB host IP, DB account, Vitis HLS path, FPGA target part

2

Execute generate-mdc.sh

Read hls-env.conf, replace the {{variables}} of the three .mdc-templates with actual values, and output to .cursor/rules/

3

Install Cursor (Windows)

Download the Cursor installation and install the Remote - SSH extension

4

SSH connection to Vitis-HLS host(HLS02、HLS03)

Open ~/cursorwork/ as workspace and ensure that the rules are located under .cursor/rules/

5

DBeaver Online Knowledge Base

View the database through SSH Tunnel (external network) or direct connection (intranet/VPN)

→ Please see README "Developer Workflow" steps 1–5 for details.

MDC rules file structure

The three .mdc rules are all set to alwaysApply: true, and are automatically loaded during Cursor dialogue.

hls-core.mdc

Section 1–5

1. Document version and overview - API endpoint quick check, Best Practice
2. Environment configuration and verification—API URL automatic detection, Vitis HLS confirmation
3. Database connection specification — get_database_url() function
4. Knowledge Base rule system - operating principles, success determination, administrator exclusive
5. Query strategy – two-track query (Track 1 + Track 2)

hls-code-standards.mdc

Section 6

6. Code snapshot management — file header / pragma comment / code_snapshot format / Comment Quality Checklist (16 items)

hls-recording.mdc

Section 7

7. Design iteration record — Front-Capture / Bind Rules / previous_ii baseline / complete_iteration API / reproduce iteration

→ Generated by generate-mdc.sh + hls-env.conf; see README "Developer Workflow" step 2 for details

Task Cheat Sheet — Task → Section

Map each task to the right MDC section and API endpoint

When you need to...	Section	Key content / endpoints
Confirm the machine environment and select the API URL	Section 2	`hostname/IP auto-detection`
Query rules + similar designs before synthesis	Section 4+5	`/api/rules/effective + /api/design/similar`
Call GET /api/rules/categories before two-track query	Section 5	`/api/rules/categories`
Pragma comments (always) + file header (after successful synthesis)	Section 6	`Three-line pragma comments + measured values in the header after csim+csynth succeed`
Confirm code_snapshot quality	Section 6	`Comment Quality Checklist (16 items)`
Resolve project_id before recording (mandatory)	Section 7	`/api/projects?type= + project_name comparison`
Record iteration / 409 / reproduce / check progress	Section 7	`complete_iteration / existing_project_id / code / progress`
rollback / backup / reset	Section 4	`Always respond: "Requires system administrator authorization"`

Best Practice (enforced compliance)

ALWAYS Query KB for rules and similar designs before designing

ALWAYS Create optimization_summary.md locally

ALWAYS After successful synthesis, record via complete_iteration API

`iteration_number` Automatically calculated by API

→ *See hls-core.mdc Section 1 "Quick Navigation" "Best Practice"*

API endpoint quick review

List of FastAPI endpoints grouped by purpose Query and Write only

Query ·GET endpoint (read)		
GET	/health	API service and database connection status (healthy/unhealthy)
GET	/api/projects?type={type}	List projects; get the only source of project_id before recording
GET	/api/projects/{project_id}	Get single project details
GET	/api/design/similar?project_type={type}	Query similar designs (Track 1); ascending order by ii_achieved; target_ii filter can be added
GET	/api/design/{iteration_id}/code	Get iteration complete code snapshot (code_snapshot / cursor_reasoning / pragmas_used)
GET	/api/rules/effective	Query rules and performance statistics (Track 2); rule_type / category / project_type / min_success_rate
GET	/api/rules/categories	Get the latest category list (required before two-track query)
GET	/api/analytics/project/{project_id}/progress	Iteration progress and history (total_iterations, ii_improvement, resource_usage)
Write ·POST endpoint (write)		
POST	/api/design/complete_iteration	The only write entry: project creation + iteration record + synthesis result + rule statistics + rollback completed in one go; iteration_number is automatically calculated by API
POST	/api/projects	Create a new project; name within the same type must be unique; duplicate → 409 Conflict + existing_project_id (recall with this ID)

→ *See hls-core.mdc Section 1 "API Endpoints" "API Response Supplement"*

Environment verification and API configuration

The environment must be verified before each HLS task (mandatory per MDC Section 2)

API URL auto-detection

On the KB host:
→ `http://localhost:8000`

On other machines:
→ `http://{KB_HOST_IP}:8000`

Automatic detection based on hostname/IP

Verification checklist

✓ Hostname/IP address
✓ Is Vitis HLS available?
✓ KB API connection status
✓ curl/jq/python3 tools

If Vitis HLS is available → run synthesis on this host
If Vitis HLS is unavailable → inform the user

Database scripts: do not hard-code DB hosts; use get_database_url() for automatic environment detection.

→ *See hls-core.mdc Section 2–3*

Backup, rollback、Data managementand access principles

The following operations are exclusive to system administrators. Users No permission implement

Backup and recovery

Backup full SQL dump + JSON metadata
Restore will overwrite the current DB

Accurate rollback

logger generates YAML → rollback execution
Two stages: preview first → then execute
_rollback_info Accurately restore statistics

Reset database

Clear data and retain schema
Complete rebuild with setup.sh

Knowledge Base access principles

operate	executor
Query rules/design	Cursor HLS users
Write new iteration	Cursor call complete_iteration automatically completed
Rollback/Backup/Reset	system administrator

→ See README "Backup and Recovery""Reset database""Rollback mechanism""Knowledge Base Access Policy"And hls-core.mdc Section 4 "Administrator Exclusive Operation"

Two-Track Query Strategy

Run before each design iteration—the two tracks are complementary

Track 1: Query best iterations

`API: /api/design/similar`

Query best historical designs with the same project_type

Results are sorted in ascending order by ii_achieved

Take approach_description / pragmas_used / cursor_reasoning

When needed, use iteration_id with GET /api/design/{iteration_id}/code

Purpose: Understand the performance ceiling for this project_type

Track 2: Effective rules (rules + statistics)

`API: /api/rules/effective`

GET /api/rules/categories — category list (like pipeline, memory, fir, cordic ...)

(Optional) Step 0: If the application category matches (e.g. fir), query application-scoped rules first

Path A: Track 1 has results → infer categories

Path B: No similar designs → fallback through Semantic filtering on rule_text → statistical sort (success_count / times_applied / priority) → compare rule priority — merge candidates from Step 0, Path A / Path B

Purpose: Retrieve on-topic rules with statistics-backed sorting

→ See hls-core.mdc Section 5 and README "Two-Track Query Strategy"

Two-tier rule system

Official Rules R### (287)

Source: UG1399 Best Practices

Example: R035 Always apply PIPELINE to innermost loop

Technology Category: dataflow, pipeline, memory, interface, optimization, synthesis, etc.

User-defined P### (100+ items)

Source: Project Optimization Experience (Extensible)

Example: P068 Combine shifts and calculation loops to eliminate dependencies

Application Category: fir, systolic, cordic, etc. (manual annotation)

Rule Priority automatically set

Priority	Keywords	Semantic
9	always, must, critical, never	Mandatory
7	do not, avoid, ensure, should	Strongly recommended
5	consider, may, recommend, prefer	Suggestive
4	(No matching keywords)	default value

→ *See hls-core.mdc Section 4 "Knowledge Base Rule System" and README "Rule Classification"*

Rule classification and manual annotation

Category is manually marked in the rule file with # Category: <name>, which is read directly by import_rules.py

type	Category	illustrate
Technical orientation	pipeline, memory, dataflow, interface, optimization, synthesis ...	General HLS best practices, included in both rule files
Application oriented	fir, systolic, cordic ...	Only rules_user_defined.txt (manual annotation)

How to use Cursor

1. Before each two-track query, call /api/rules/categories to obtain the latest list
2. Infer category based on user input (such as "FIR filter, target II=1" → fir + pipeline)
3. API queries are automatically merged and returned, regardless of official / user_defined
4. To add a new category, you only need to mark it in the rules file, and import_rules.py does not need to be modified.

→ See README "Manual Category Annotations" "Rule classification"

Design iteration workflow

Before design

- Front-Capture: Scan message maintenance USER_REF_CODE / USER_SPEC
- Two-Track query KB (Track 1 + Track 2)
- Bind Rules: Bind selected rules to APPLIED_RULES
- Determine previous_ii baseline

In practice

- Write code + pragma comment (do not write file header)
- Only .h / .cpp / *_tb.cpp / .inc files are allowed

After synthesis

- Only record if csim + csynth are successful
- Analysis report → fill in header comment (actual value)
- Query project_id → complete_iteration
- Update local markdown document backup

II Not up to standard → Based on this result, select the best solution from the pragma combinations that have not yet been tried, and return to "before design" to iterate again

→ *See hls-core.mdc Section 4 + hls-recording.mdc Section 7 "Main Flow"*

II Indicators and Success Determination

Layer A — KB statistical basis

`ii_bneck`
= max(Interval of all pipeline loops)

`success = (current_ii < previous_ii)`
Both ii_bneck

Driver success_count, avg_ii_improvement
rules_effectiveness The only basis for statistics

Layer B — Description added

`ii_top`
= Interval for top-level function lines

Write cursor_reasoning + local document

Only ii_top / latency improved but ii_bneck did not drop:
success cannot be set to true

Avoid confusing pipeline rule effects

previous_ii baseline: Layer 1 (measured) → Layer 2 (estimated / fallback)

The first iteration uses Cursor to estimate the theoretical baseline (such as FIR shift_reg[128] → previous_ii = 128); subsequent iterations use the previous round of measured ii_bneck

→ See hls-recording.mdc Section 7 "II Metric Priority" "previous_ii Baseline Definition"

Code Snapshot Specification

delimited format

```
// === {filename} ===
```

Fixed order:
`[.h] → .cpp → *_tb.cpp → [.inc]`

.h / .inc are optional (include if available)
testbench must be the version passed by csim
The delimited line itself is not part of the file content

Two-stage writing

When writing code
→ pragma comment (three lines above each #pragma HLS)
→ Technique / Parameters / Rationale
→ Do not write file header comment

After successful synthesis
→ Fill in the complete header (actual value) at one time
→ Synthesis Result / Resources / Applied Rules
→ All values come from csynth reports

Comment Quality Checklist (16 items) — all must pass before calling complete_iteration

→ *See hls-code-standards.mdc Section 6*

Iteration records complete_iteration

The only write entry - complete project creation + iteration records + synthesis results + rule statistics + rollback information in one go

`project_id`	GET /api/projects + project_name exact comparison (sole source)
`code_snapshot`	Contains header comment (actual value), passes Checklist
`synthesis_result`	ii_achieved (= ii_bneck)、latency、timing、resources
`rules_applied[]`	rule_code + previous_ii + current_ii + success
`cursor_reasoning`	ii_bneck / ii_top before and after comparison + basis for success determination

Error handling

409 Conflict → Retrieve existing_project_id Recall | iteration_number automatically calculated by API

→ See hls-recording.mdc Section 7 "Post-Synthesis" Step 2–4 and README "API Endpoint Reference"

Reproduce and Troubleshooting

Reproduce the specified iteration

1. Get iteration_id (progress / similar)
2. GET /api/design/{id}/code to retrieve code_snapshot
3. Restore files by separated lines
4. Automatically generate run_hls.tcl
5. vitis_hls -f run_hls.tcl execute and verify

Only used for verification/learning, does not trigger automatic recording

Troubleshooting common obstacles

DBeaver can't connect?

→ SSH tunnel interrupted, rebuilt

→ docker ps check PG container

Cursor Insufficient permissions?

→ Switch to Agent mode

→ Restart Remote SSH

API returns empty value?

→ project_type case (fir ≠ FIR)

→ min_success_rate is set to 0 (not set success rate threshold)

→ See README "Troubleshooting" and hls-recording.mdc "Reproduce Specific Iteration"

Demo example overview

6 practical Cursor + Vitis HLS design processes showing how KB works in different design types

example	design	Iterate	model	focus
Systolic	4×4 output-stationary systolic GEMM	3	A	One prompt i.e. II=1; 5 P-rules
Matmul3×3	3×3 matrix multiplication FIFO interface	3	A	DATAFLOW overlaps 3 matrices
FIR128	128-tap FIR streaming filter	3	A	BRAM circular buffer → 2-way parallel MAC
CORDIC	sin/cos rotation (0 DSP)	3	A	DATAFLOW vs PIPELINE vs Time Sharing
DFT-16	16-point discrete Fourier transform	4	B	II vs timing design space; 3 drafting rules
FFT-16	16-point fast Fourier transform	2	B	Ping-pong + template banks; 4 draft rules

Mode A — mature P-rules

The user provides specification description, and KB returns complete rules. Iter 1 equals II=1.

Mode B - fewer P-rules, requires architecture guidance

Users need to provide architectural decisions in subsequent iterations, and draft rules will be produced in the process (to be upgraded to P###)

Each example II=1 reaches the analysis

The judgment basis for II=1 is ii_bneck (max of pipeline loop Interval), which is consistent with the MDC rule rules_applied.success

Pattern A — iter 1 equals II=1 (4 examples)

example	ii_bneck	iter 1 clock / timing	Subsequent iteration direction
Systolic	1 ✓	10 ns — timing failed (iter 2 → 12 ns)	clock relaxation → MAC path decomposition experiment
Matmul3×3	1 ✓	20 ns — timing met	Subsequent optimization of latency and throughput
FIR128	1 ✓	12 ns — timing met	Change streaming mode and improve parallelism
CORDIC	1 ✓	10 ns — timing met	Comparison of three architectural styles (0 DSP in the whole process)

KB has mature P-rules → ii_bneck=1 in place from the first iteration, and subsequent focus on timing / latency / throughput / area

Mode B — II=1 requires multiple iterations or deliberate trade-offs (2 examples)

example	iter 1	final	process and reason
FFT-16	2	1 ✓	iter 1 in-place RAW dependency → II=2; iter 2 reaches II=1 after four structural modifications
DFT-16	1	2	iter 1 reaches II=1 but timing fails; deliberately retreats to II=2 + 12 ns in exchange for timing closure

in conclusion

- **FFT**— The only example of "II=1 achieved through multiple iterations" (iter 2's four structural transformations were solved simultaneously)
- **DFT**— The only example of “final ii_bneck ≠ 1” (II=2 is a deliberate choice — the timing of II=1 cannot be closed)

Key operation tips and Cursor behavior observation

Confirm that the rules have been loaded

For new environments, you can enter "What rules are you following?" or "Summarize your active Cursor rules" to confirm that the .mdc rules have taken effect.

Tips and follow KB rules

It is recommended to explicitly add "follow KB rules" when starting to design. If not prompted, Cursor will sometimes skip KBqueries and directly use its own knowledge to design without checking the rules or recording them.

KB write behavior inconsistent

MDC stipulates that KB will be automatically written after successful synthesis. In fact, Cursor sometimes does it, sometimesit asks the user for confirmation first, and sometimes it requires the user to proactively remind "save to KB"

Specify project name and iteration

Examples often use "save this to KB {project} iter{N}" to explicitly specify it to avoid Cursor from creating the wrong project or skipping records.

Adjust one parameter at a time

In Mode B, it is recommended to change only one variable (II target, partition, clock) at a time to retain causal traceability and facilitate upgrading to formal rules.

Timing fails first to relax clock

Multiple examples show that relaxing the clock (10→12 ns) is more effective than changing the architecture. Cursor sometimesover-modifies the RTL. You can be prompted to try the clock first.

→ *Comprehensive practical experience of demo examples*

Systolic Array — 4×4 matrix multiplication

Mode A | 3 iterations | 5 P-rules → iter 1, that is, II=1

Starting prompt

```
Design a systolic array accelerator for matrix multiplication (C = A × B)
using output-stationary architecture, SIZE×SIZE PEs, AXI stream.
Project named Systolic_Demo, please follow KB design rules.
```

Iteration 1 — II=1 on first design (timing failure −0.90 ns)

Track 2 returns 5 P-rules: P085 (pipeline t loop only), P084 (PE array complete partition), P090 (conservative T upper bound), P094 (A row-major / B col-major), P047 (AXI I/O outside MAC loop)

Architecture: output-stationary wavefront, 16 DSP (one per PE), PIPELINE II=1 in time dimension t, space dimension (i,j) not pipeline

Result: II=1 ✓, DSP=16, LUT=4906 — but timing violation (slack −0.90 ns @ 10 ns clock)

Why does iter 1 make II=1? KB rules pre-solve four common failure reasons: data dependency (P085 wavefront), memory port conflict (P084 partition), stream blocking (P047 phase separation), and loop structure (P085 pipeline on t only).

→ *demo_systolic.md*

Systolic Array — 4×4 matrix multiplication — iterative process

Iter 2 — Clock relaxes 10→12 ns

No architectural changes, only relaxing clock → timing met (slack +0.56 ns). II=1, DSP=16 remain unchanged.

Iter 3 — MAC path decomposition (experimental)

Tip: "maintaining II=1, try decomposing the MAC critical path to improve timing"
Split multiply + accumulate into two phases + BIND_OP. Result: timing met but slack is only +0.03 ns (worse than iter 2). Conclusion: On this target component, clock relaxation is more effective than structural decomposition.

iter 1→II=1 (P-rules driver)→ iter 2 clock to relax the solution to timing → iter 3 to verify that the MAC decomposition effect is limited

Key rules: P085 P084 P090 P094 P047 (all User-defined) + R036

→ *demo_systolic.md*

Matmul 3×3 — FIFO matrix multiplication

Mode A | 3 iterations | FIFO → Area optimization → DATAFLOW overlap

Starting prompt

```
Design a Matrix Multiplier using FIFO, follow the KB rules
Matrix size: A 3×3, B 3×3, Result 3×3
Requirements: read one sample per clock cycle using FIFO interface, minimizing area
```

Iteration 1 — Minimum area FIFO baseline (II=1, DSP=1)

Track 2 return: P047 (FIFO write cannot be within MAC loop), P035 (only local 3×3 buffer), P039 (stream depth)

Architecture: four-stage sequence (load A → load B → MAC → write C), only innermost loop pipeline II=1

Single DSP MAC, ap_ctrl_hs single call

Result: Latency=73 cycles, DSP=1, LUT=1070 — timing met after clock relaxation to 20 ns

P047 prevents writing to FIFO in k accumulation loop (will destroy II=1); P035 prevents unnecessary full matrix copying

→ *demo_matmul3x3.md*

Matmul 3×3 — FIFO matrix multiplication — iterative process

Iter 2 — Panel Load + Flat Output（Latency −41%）

Tip: "optimize latency further in the row-stationary direction"
B loads first → A loads → flat ij loop (k fully UNROLL, 3 DSP parallel) → Latency 73→43 cycles, DSP 1→3, LUT −43%

Iter 3 — DATAFLOW overlapping 3 matrix

User prompts overlap timing diagrams. DATAFLOW is split into load_panels + compute_all, internal FIFO depth=64 (P039).
Single matrix valid ~18-24 cycles (after overlap), DSP=3 unchanged. P039 prevents deadlock caused by insufficient FIFO depth.

correctness first (iter 1) → single latency optimization (iter 2) → multi-matrix throughput (iter 3 DATAFLOW)

Key rules: P047 P035 P039 (User-defined) + R035/R036 R051 R001

→ *demo_matmul3x3.md*

FIR128 — 128-tap streaming filter

Mode A | 3 iterations | Streaming → Continuous mode → 2-way parallel MAC

Starting prompt

```
Design a 128-tap FIR filter with AXI Streaming interface using a streaming
loop architecture, with minimal resource usage. Name FIR128_Demo, follow KB rules.
```

Iteration 1 — Area minimum streaming baseline (II=1, DSP=1, BRAM=2)

Track 2 return: P047 (stream I/O cannot be in MAC loop), P079 (circular delay line in BRAM)

Architecture: sample_loop (read and write AXI-Stream) + tap_mac (k=0..127, PIPELINE II=1), single DSP MAC
delay_line[128] is stored in BRAM (P079) as circular buffer to avoid 128-tap shift register

Result: II=1, Per-sample ~138 cycles, DSP=1, BRAM=2, LUT=295 — 12 ns timing met

P047 is the most critical rule: the natural tendency is to put axis.read() at the top of the MAC loop, which will immediately destroy II=1

→ *demo_fir128.md*

FIR128 — 128-tap streaming filter — iterative process

Iter 2 — Continuous streaming (ap_ctrl_none)

Tip: "outer streaming loop, no NUM_SAMPLES, no fixed packet length"
ap_ctrl_hs → ap_ctrl_none, while(1) runs forever. Performance remains unchanged (~138 cyc/sample), LUT decreases slightly.

Iter 3 — 2-Way Parallel MAC (Throughput ×1.9)

Tip: "keep ii=1/DSP limit 2/more MAC, try to improve throughput"
k += 2, two DSP parallel MAC. delay_line + coeff_rom is changed to complete partition (BRAM→LUT).
Per-sample 138→72 cycles, DSP=2, BRAM 2→0, LUT 259→4199 (full partition cost)

Smallest area (iter 1) → true streaming (iter 2 ap_ctrl_none) → doubled throughput (iter 3 2-way MAC)

Key rules: P047 P079 (User-defined) + R035/R036

→ *demo_fir128.md*

CORDIC — sin/cos rotation (0 DSP)

Pattern A | 3 iterations | Comparison of three architectural styles: DATAFLOW vs PIPELINE vs time sharing

Starting prompt

```
Design a CORDIC and name it Cordic_Demo, follow KB rules
Function: rotation mode | Input: angle θ | Output: sin(θ), cos(θ)
Numeric: ap_fixed<16,2> | Algorithm: 16 iterations | Constraints: shift-add only
```

Iteration 1 — DATAFLOW full pipeline (II=1, 0 DSP, BRAM=51)

Track 2 return: P071 (shift-add microrotation, disable DSP), P072 (atan precomputed table partition)

Architecture: 18 DATAFLOW processes + 17 hls::stream FIFO, independent pipeline for each stage

Pure shift-add operation (P071), 0 DSP — left to other accelerators in the system

Result: II=1, Latency=49 cycles, DSP=0, BRAM=51, LUT=5073

The BRAM is high because each of the 17 stream FIFOs (depth=32) occupies 3 BRAM

P071 is an architectural cornerstone: CORDIC math only requires shift-add, but without clear rules HLS or developers may misuse multipliers

→ *demo_cordic.md*

CORDIC — sin/cos rotation (0 DSP) — iterative process

Iter 2 — Register Pipeline（BRAM 51→0, Latency −65%）

Tip: "Reduce BRAM by eliminating hls::stream FIFOs, prefer register-based pipeline"
Remove DATAFLOW and use single PIPELINE II=1 + 16 iterations to automatically expand to register stages.
BRAM 51→0, Latency 49→17, FF −69%, LUT −41%. Likewise II=1, 0 DSP.

Iter 3 — Time Sharing (LUT −78%, Top II=23)

Tip: "reduce LUT by sharing CORDIC stages, keep II=1, allow slight latency increase"
UNROLL factor=1 prevents space expansion → 1 shared microrotation unit is reused 16 times.
LUT 2977→662, FF 1344→152, but Top II=1→23 (area vs throughput trade-off)

DATAFLOW (throughput priority) → single PIPELINE (best balance) → time sharing (area priority), 0 DSP throughout

Key rules: P071 P072 (User-defined) + R035/R036

→ *demo_cordic.md*

DFT-16 — Discrete Fourier Transform

Mode B | 4 iterations | II vs timing design space exploration + 3 draft rules

Starting prompt

```
Make a DFT that takes N=16 complex inputs and outputs N=16 results.
Use fixed-point numbers and a simple double loop with pipeline.
Follow KB rules, save to DFT_Demo project.
```

Iteration 1 — Maximum parallelism (II=1, timing failed −0.84 ns)

Track 2 return: P003 only (partition depth control load/compute overlap) - P-rules limited, expected mode B

Architecture: complete partition all arrays + PIPELINE II=1 in inner n loop

Result: II=1 ✓ But timing violation (slack −0.84 ns @ 10 ns), LUT=3380, BRAM=0

P003 Guiding Principle: Partition depth determines II, but complete partition causes timing problems in this design

Mode B: There is only one P-rules, which cannot be implemented at one time. The user needs to guide the direction of subsequent iterations.

→ *demo_dft.md*

DFT-16 — Discrete Fourier Transform — Iterative Process

Iter 2 — reduce parallelism priority timing (II=4, 15 ns)

Tip: "reduce parallel operations, even if it slightly increases latency or II"
Remove all partitions → BRAM access, clock 15 ns. II=4, timing met, LUT 3380→526.
Draft rules: throughput ↔ timing trade-off — Sometimes it is necessary to accept II regression in exchange for timing closure

Iter 3 — Balance: cyclic/2 + II=2 (timing still fails)

Tip: "try cyclic factor=2, target II=2, avoid full partition"
cyclic factor=2 → II=2, but timing is still −0.84 ns @ 10 ns. Draft rules: HLS estimate vs backend flow

Iter 4 — Final: same architecture + 12 ns → timing met ✓

Tip: "use the above configuration, clock fixed at 12 ns". The RTL is unchanged, only the clock is relaxed.
II=2, timing met (+0.27 ns). Draft rule: clock-only tuning when II unchanged

iter 1 max II but timing fails → iter 2 sacrifices II to fix timing → iter 3-4 find balance point (II=2, 12 ns)

Key rules: P003 (User-defined) + R035/R036 + 3 draft rules (to be upgraded P###)

→ *demo_dft.md*

FFT-16 — Fast Fourier Transform

Mode B | 2 iterations | 4 structural modifications up to II=1 + 4 draft rules

Starting prompt

```
Design a FFT_Demo project following KB rules, an FFT (N=16) with complex inputs,
multiple stages, fixed-point, pipeline each stage without fully unrolling.
Precomputed twiddle factors, good performance without excessive resources.
```

Iteration 1 — Radix-2 DIT baseline (II=2, 25 ns timing met)

Track 2 postback: P035 only (no full-buffer memcpy between stages) - limited P-rules, expected mode B

Architecture: 4 stage loops each with PIPELINE II=1, in-place butterfly update

Result: II=2 (in-place RAW dependency causes II > 1), Latency=116, DSP=8

The clock is relaxed to 25 ns before timing met. LUT=3723

In-place update mode (buf[i] = f(buf[i], buf[j])) produces a cross-iteration read-after-write dependency → II cannot be 1

→ *demo_fft.md*

FFT-16 — Fast Fourier Transform — Iterative Process

Iter 2 — Four structural modifications → II=1 @ 10 ns

Tip: "Break butterfly into multiple pipeline stages, resolve data dependency using buffering"

Four transformations must be completed at the same time (there are still bottlenecks in doing each one individually):
1. Ping-pong double buffer — Eliminate in-place RAW dependency (Draft Rule 1)
2. Template-fixed banks — banks determined at compile time → eliminate sparsemux (draft rule 2)
3. ARRAY_PARTITION dim=2 complete — resolves HLS 200-880 double write conflict (draft rule 3)
4. BIND_OP + split MAC — DSP binding + staged multiply/add → 10 ns timing met (draft rule 4)

Result: II=1 ✓, clock 10 ns timing met, DSP 8→16, LUT 3723→7427
Wall-clock 2× faster（1440 ns vs 2900 ns）

iter 1 baseline II=2 (in-place dependency) → iter 2 four structural transformations achieved simultaneously II=1

Key rules: P035 (User-defined) + R035/R036 R074 + 4 draft rules (to be upgraded P###)

→ *demo_fft.md*

cursor-hls-kb

Cursor AI × Vitis HLS × Knowledge Base

GitHub: github.com/aicoforge/cursor-hls-kb